

Uyku Kalesi: Nesneye Yönelik Programlama ile Temalı Bir Kule Savunma Oyununun Tasarımı ve Simülasyonu

Burak Kılıcı

Bilgisayar Mühendisliği Bölümü

Kocaeli Üniversitesi

Kocaeli, Türkiye

Öğrenci No: 230202111

230202111@kocaeli.edu.tr

Özet—Bu çalışmada, Programlama Laboratuvarı-I dersi kapsamında Java dili ve Swing kütüphanesi kullanılarak *Uyku Kalesi* adlı, gerçek zamanlı ve temalı bir kule savunma oyunu geliştirilmiştir. Oyunda bir çocuğun yatak odasına gece yarısı dolaptan çıkan kabuslar, kesintisiz bir yol boyunca yatağa ilerlemekte; oyuncu ise sınırlı bir bütçe (şeker) ile yerleştirdiği oyuncak savunucularla çocuğun uykusunu korumaya çalışmaktadır. Proje isterlerindeki üç düşman ve üç kule türü; soyut sınıflar, kalıtım, çok biçimlilik ve kapsülleme ilkelerine dayanan iki ayrı sınıf hiyerarşisi ile modellenmiştir. Simülasyon motoru grafik arayüzden tamamen bağımsız tasarlanmış; arayüz ile motor arasındaki haberleşme komut çağrıları, sabit aralıklı bir oyun döngüsü ve Gözlemci (Observer) deseni ile sağlanmıştır. Tüm oyun olayları zaman damgalı olarak `savunma_gunlugu.txt` dosyasına kaydedilmektedir. Savunmasız, dengeli ve yalnızca topçu kulesi içeren üç senaryoda yapılan simülasyonlar; zırh formülünün, hedef önceliğinin, alan hasarının ve yavaşlatma etkisinin istelere uygun çalıştığını, oyunun hem kazanma hem de kaybetme koşullarıyla tutarlı biçimde sonuçlandığını göstermiştir.

Anahtar Kelimeler—kule savunma, nesneye yönelik programlama, Java Swing, oyun döngüsü, gözlemci deseni, simülasyon

I. GİRİŞ

Kule savunma (*tower defense*) oyunları, oyuncunun önceden belirlenmiş bir yol boyunca ilerleyen düşman dalgalarını, yolun çevresine yerleştirdiği savunma birimleriyle durdurmaya çalıştığı strateji oyunlarıdır. Türün temel gerilimi; sınırlı kaynağın hangi birime, hangi konuma ve hangi zamanda harcanacağı kararından doğar [2]. Bu yapı, birbirinden farklı davranışlara sahip ancak ortak özellikleri paylaşan çok sayıda nesnenin aynı anda etkileştiği bir ortam sunduğu için nesneye yönelik programlama (NYP) ilkelerini uygulamak açısından elverişli bir problem alanıdır.

Proje dokümanı [1]; üç düşman türü (standart, zırhlı, uçan), üç kule türü (okçu, topçu, buz), iki dalgalı bir oyun akışı, zırhı hesaba katan bir hasar formülü, canlı bir grafik arayüz ve zaman damgalı bir olay kayıt dosyası istemektedir. Ayrıca her grubun özgün bir tema seçmesi ve bu temanın sınıf, değişken ve metot isimlerinden arayüze ve kayıt dosyasına kadar her katmana yansıtılması beklenmektedir.

Bu raporda geliştirilen *Uyku Kalesi* oyunu anlatılmaktadır. Çalışmanın başlıca katkıları şunlardır: (i) düşman ve savunucular için soyut sınıf temelli, genişletilebilir iki sınıf hiyerarşisi; (ii) Swing'e hiç bağımlı olmayan, tek başına test edilebilen bir simülasyon motoru; (iii) arayüz ile motor arasında komut, döngü ve olay bildirimi olmak üzere üç kanallı, gevşek bağlı bir haberleşme modeli; (iv) her hasar hesabını adım adım gösteren, izlenebilir bir kayıt sistemi; (v) temayı yalnızca görselde değil, oyun mekaniğinin kendisinde (ör. *ıslatma* ile yavaşlatma) yaşatan bir tasarım.

Raporun geri kalanı şöyle düzenlenmiştir: Bölüm II'de mimari, sınıf diyagramları, simülasyon motoru ve arayüz-motor haberleşmesi; Bölüm III'te oyun konsepti ve isimlendirme stratejisi; Bölüm IV'te simülasyon sonuçları, kayıt çıktıları ve ekran görüntüleri sunulmakta; Bölüm V'te ise sonuçlar değerlendirilmektedir.

II. YÖNTEM

A. Geliştirme Ortamı ve Gereksinimler

Oyun Java 11 (ve üzeri) sürümüyle uyumlu olarak yazılmış, grafik arayüz için standart kütüphanedeki Swing [6] kullanılmıştır; projede harici bir kütüphane bulunmamaktadır. Kaynak kod dört paketten ve 31 sınıf/arayüzden oluşmakta, yaklaşık 3800 satırdır. Türkçe karakterler nedeniyle tüm dosyalar UTF-8 olarak kaydedilmiş, derleme `javac -encoding UTF-8` ile yapılmıştır. Sınıf diyagramları PlantUML [10] ile UML 2.5 gösterimine [9] uygun biçimde çizilmiştir.

Dokümandaki sayısal isterler Tablo I ve Tablo II'da özetlenmiştir. Zırhlı düşmanın canı standart düşmanın %50 fazlası, hızı %50 eksiktir; uçan düşmanın hızı ise %50 fazladır. Zırh değeri her zırhlı düşman için oyun başında [50, 100] aralığında rastgele bir tam sayı olarak belirlenmektedir.

B. Katmanlı Mimari ve Paket Yapısı

Kod, sorumlulukların ayrılması ilkesine göre dört pakete bölünmüştür:

- `uykukalesi.varlik`: Oyun dünyasındaki tüm nesneler (kabuslar, savunucular, mermi, görsel efektler) ile `Cizilebilir` ve `SavasAlani` arayüzleri.

Tablo I
KABUS (DÜŞMAN) TÜRLERİNİN PARAMETRELERİ

Tür	Korku enerjisi	Hız (px/sn)	Zırh	Ödül (şeker)	Uyku hasarı	Yol süresi
Toz Canavarı	50	50	0	10	5	39,6 sn
Teneke Robot	75	25	50-100	20	10	79,3 sn
Gece Kelebeği	50	75	0	15	5	26,4 sn

Yol süresi: 1982 px uzunluğundaki yolun yavaşlatılmadan geçilme süresi.

Tablo II
OYUNCAK SAVUNUCU (KULE) TÜRLERİNİN PARAMETRELERİ

Tür	Vuruş	Arahk	Menzil	Maliyet	Özel davranış
Lastik Sapan	10	1 sn	140 px	50	Zırhlıya %50 az
Bilye Mancınığı	20	3 sn	130 px	75	50 px alan; uçana yok
Su Tabancası	15	2 sn	120 px	70	3 sn %50 yavaşlatma

- `uykukalesi.motor`: Simülasyon motoru, harita, yol, oyuncak yuvaları, akın yöneticisi ve savunucu fabrikası. Bu paket hiçbir `Swing` sınıfı kullanmaz.
- `uykukalesi.kayıt`: Zaman damgalı kayıt dosyasının yazımı ve sayı biçimlendirme.
- `uykukalesi.arayüz`: Ana pencere, ana menü, oyun ekranı, harita paneli ve özel çizimli düğmeler.

Bu ayırım sayesinde motor, arayüz olmadan bir test sınıfından çalıştırılabilmiş ve Bölüm IV'teki senaryolar bu yolla tekrarlanabilir biçimde doğrulanmıştır.

C. Sınıf Diyagramları

Şekil 1, oyun varlıklarının sınıf hiyerarşisini göstermektedir. En üstte yer alan soyut `OyunVarligi` sınıfı kimlik ve konum bilgisini, `Cizilebilir` arayüzü ise ekrana çizilebilme sözleşmesini tanımlar. Bu sınıftan iki soyut dal türetilir: düşmanlar için `KabusYaratigi`, kuleler için `OyuncakSavunucu`. Somut türler (`TozCanavari`, `TenekeRobot`, `GeceKelebegi`, `LastikSapan`, `BilyeMancinigi`, `SuTabancasi`) yalnızca kendilerini farklı kılan davranışları gerçekleştirir.

Şekil 2 ise simülasyon motoru, arayüz ve kayıt paketlerindeki sınıflar arasındaki ilişkileri göstermektedir. `SimulasyonMotoru`, `AkinYoneticisi`'ni bileşim (*composition*) ilişkisiyle içerir; `Harita` ise yolu ve on oyuncak yuvasını barındırır. Arayüz tarafındaki `OyunEkranı`, motora komut gönderir ve `OyunOlayDinleyicisi` arayüzünü gerçekleştirerek motordan bildirim alır.

D. Nesneye Yönelik Programlama İlkelerinin Uygulanması

Soyutlama: `OyunVarligi`, `KabusYaratigi` ve `OyuncakSavunucu` soyut sınıflardır ve doğrudan örneklenebilir. Örneğin `getSekerOdulu()`, `getUykuHasari()`, `ucabilirMi()` ve `isabetEt()` soyut metotlardır; her alt sınıf bunları gerçekleştirmek zorundadır. Motor, savunucuları yalnızca `SavasAlani` arayüzü üzerinden tanır; savunucu sınıfları motorun somut sınıfını bilmez.

Kalıtım: Can, hız, zırh, menzil ve atış aralığı gibi ortak alanlar ile hareket, hasar alma, hedef seçme ve çizim gibi ortak

davranışlar üst sınıfta bir kez yazılmıştır. Böylece yeni bir düşman ya da kule türü eklemek, yalnızca farklı olan birkaç metodu yazmaktan ibaret hâle gelmiştir.

Çok biçimlilik: Motor, `List<OyuncakSavunucu>` üzerinde dolaşarak her savunucunun `guncelle()` metodunu çağırır; savunucunun türünü bilmez. Davranış farkları metot ezme (*overriding*) ile sağlanır: `LastikSapan`, `tabanHasarHesapla()` metodunu ezerek zırhlı hedefe %50 ceza uygular; `BilyeMancinigi`, `hedefAlabilirMi()` metodunu ezerek uçanları eler; her türün `isabetEt()` metodu farklıdır (tek hedef, alan hasarı, ıslatma). `Harita` paneli de her varlık için yalnızca `ciz()` çağırır. `SavunucuTuru` enum'ındaki her sabit `uret()` metodunu farklı biçimde gerçekleştirerek bir fabrika görevi görür.

Kapsülleme: Tüm alanlar `private` tanımlanmıştır. Bir kabusun korku enerjisi yalnızca `hasarAl()` ile azalabilir; bu metot enerjinin negatife düşmesini engeller ve enerji sıfıra ulaştığında durumu `DAGILDI` yapar. Hız yalnızca `islat()` ile geçici olarak düşer; şeker ve uyku değerleri ise yalnızca motorun içinden değiştirilir. Arayüz bu değerleri salt okunur erişim metotlarıyla okur [5].

Şablon metot: `OyuncakSavunucu.guncelle()` final olarak tanımlanmış ve her karede izlenecek sırayı (doldurma süresi → hedef seçimi → fırlatma) sabitlemiştir (Kod 1). Alt sınıflar bu sırayı bozamaz; yalnızca `hedefSec()`, `hedefAlabilirMi()` ve `isabetEt()` adımlarını özelleştirebilir [3].

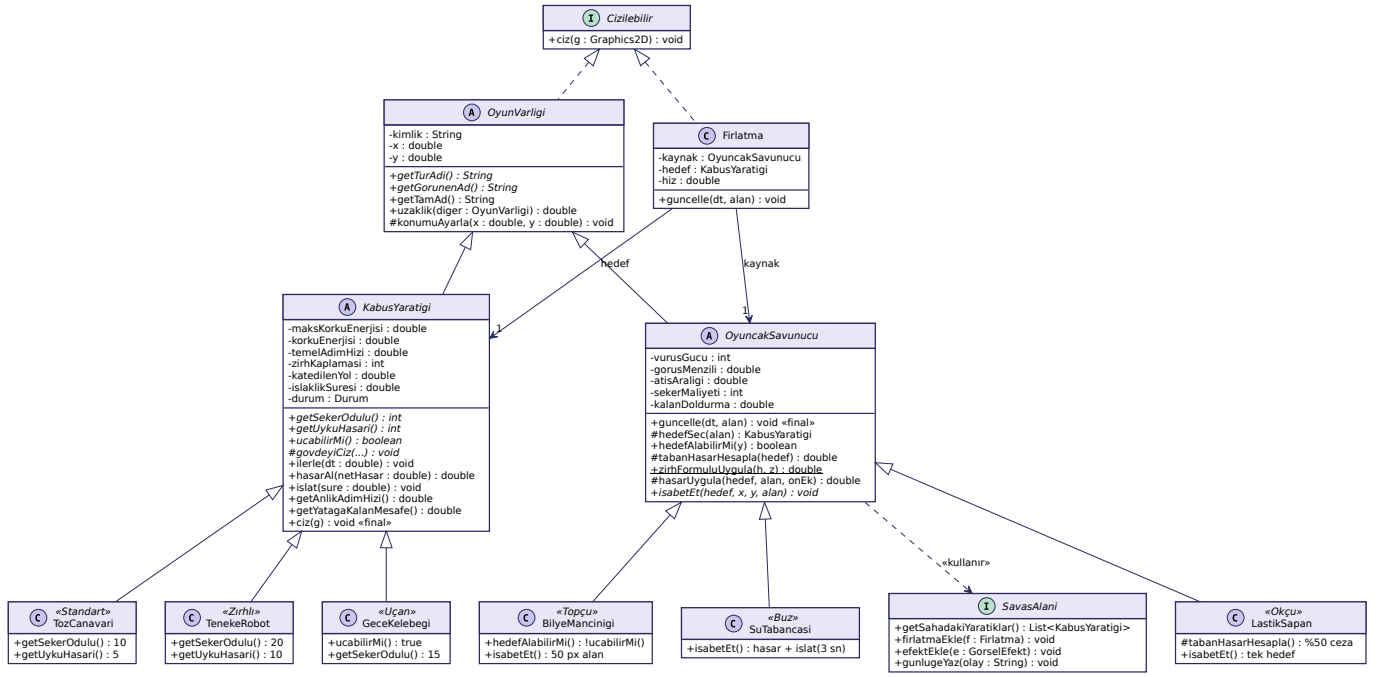
Kod 1. Savunucuların şablon metodu (kısaltılmış)

```
1 public final void guncelle(double dt, SavasAlani alan) {
2     if (kalanDoldurma > 0) {
3         kalanDoldurma -= dt;
4         if (kalanDoldurma > 0) return; // henüz dolmadı
5     }
6     KabusYaratigi hedef = hedefSec(alan); // alt sınıf
7     if (hedef == null) { kalanDoldurma = 0; return; }
8     kalanDoldurma = atisAraligi;
9     alan.firlatmaEkle(new Firlatma(this, hedef,
10         getFirlatmaHizi()));
11 }
```

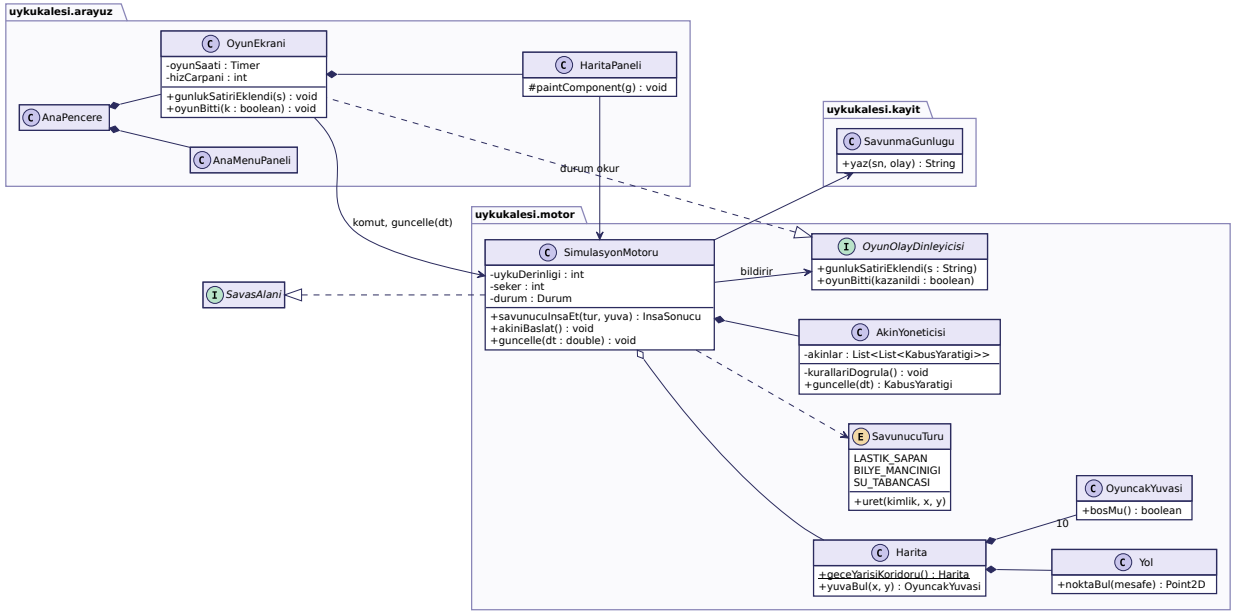
E. Simülasyon Motoru

1) Durum makinesi: Motorun yaşam döngüsü Şekil 3'deki beş durumlu makine ile yönetilir. Oyun `HAZIRLIK` durumunda başlar; bu sırada oyuncu savunucu kurabilir. İlk akın oyuncunun düğmeye basmasıyla başlar. Bir akın temizlendiğinde sonraki akına kadar 10 saniyelik bir `AKIN_ARASI` geçer; oyuncu isterse bu süreyi beklemeden ikinci akını başlatabilir. `Uyku` derinliği sıfıra indiğinde `KAYBEDILDI`, son akındaki tüm kabuslar temizlendiğinde ise `KAZANILDI` durumuna geçilir.

2) Oyun döngüsü: Motorun `guncelle(dt)` metodu her çağrıldığında sırasıyla şu adımları işletir [4]: (1) akın akışı (gerekirse dolaptan yeni bir kabus çıkarılır; iki çıkış arası 1,8 sn), (2) sahadaki kabusların ilerletilmesi, (3) savunucuların güncellenmesi, (4) mermilerin uçurulması ve isabet anında hasar uygulanması, (5) dağılan ve yatağa ulaşan kabusların işlenmesi, (6) kazanma/kaybetme denetimi. Hasar, mermi



Şekil 1. Oyun varlıklarının UML sınıf diyagramı. Soyut sınıflar ve soyut metotlar italik, final şablon metotlar «final» ile işaretlenmiştir.



Şekil 2. Simülasyon motoru, grafik arayüz ve kayıt paketlerinin UML sınıf diyagramı.

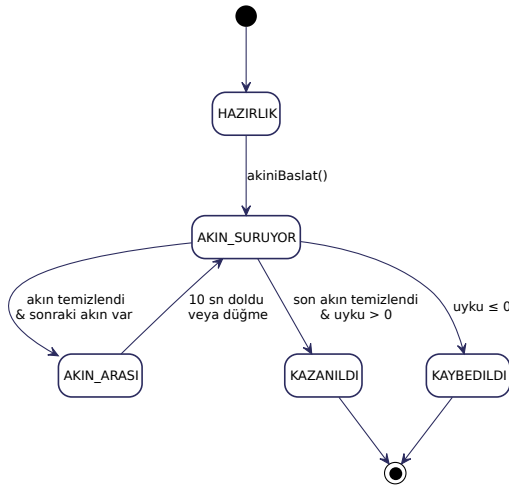
atıldığı anda değil hedefe ulaştığında uygulanır; hedef bu sırada dağılmış ya da yatağa ulaşmışsa mermi boşa gider.

3) *Hareket modeli*: Yol, n köşe noktası $P_0, \dots, P_{n-1}$ ile tanımlanan bir çoklu doğrudur. Her köşe için birikimli uzunluk $L_k = \sum_{i < k} \|P_{i+1} - P_i\|$ önceden hesaplanır. Her kabus yalnızca kat ettiği mesafeyi (s) tutar; konumu bu mesafeden

türetilir:

$$s_{t+\Delta t} = s_t + v \Delta t, \quad P(s) = P_k + \frac{s - L_k}{L_{k+1} - L_k} (P_{k+1} - P_k), \quad (1)$$

burada k , $L_k \leq s < L_{k+1}$ koşulunu sağlayan kenardır. $s \geq L_{n-1}$ olduğunda kabus yatağa ulaşmış sayılır ve haritadan silinir. Bu model, köşelerde atlama ya da yoldan sapma olmadan, kare hızından bağımsız ve kesintisiz bir hareket sağlar.



Şekil 3. Simülasyon motorunun durum diyagramı.

Uçan kabuslar da aynı yolu izler; ancak yerden 16 px yukarıda, gölgeleriyle birlikte çizilir.

4) *Hedef seçimi:* Okçu ve buz kuleleri menzildeki hedeflenebilir kabuslar kümesi M içinden üsse en yakın olanı seçer:

$$h^* = \arg \min_{y \in M} (L_{n-1} - s_y). \quad (2)$$

Topçu kulesi için M kümesi uçan kabusları içermez. Topçunun menziline uçan bir kabus olduğunda bu durum, aynı kabus için bir kez olmak üzere kayıt dosyasına yazılır.

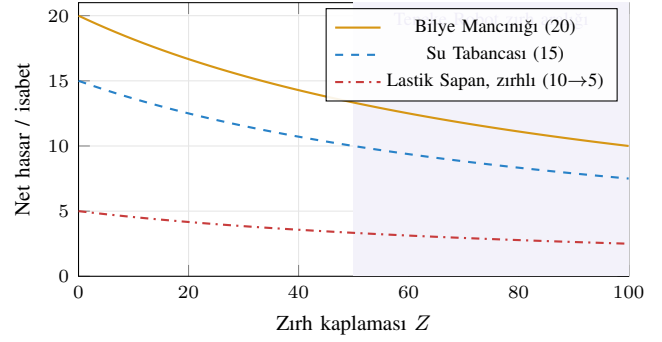
5) *Hasar ve etkiler:* Dokümandaki zırh formülü tüm savunucular için uygulanır:

$$H_{\text{net}} = H_{\text{taban}} \left(1 - \frac{Z}{Z + 100} \right) = H_{\text{taban}} \frac{100}{Z + 100}. \quad (3)$$

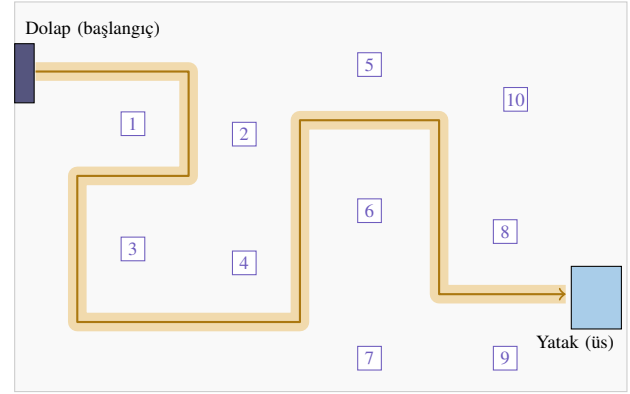
Lastik Sapan için taban hasar, hedef zırhlıysa önce %50 azaltılır: $H_{\text{taban}} = 10(1 - 0,5 \cdot \mathbb{I}_{\text{zırhlı}})$. Bu sıra, dokümandaki örnek kayıt satırıyla birebir aynıdır (Kod 2). Bilye Mancınığı isabet noktasının 50px çevresindeki tüm yer kabuslarına (uçanlar hariç) ayrı ayrı formül uygulayarak alan hasarı verir. Su Tabancası hasarın yanında hedefi 3 saniye ıslatır; ıslak kabusun anlık hızı $v = v_0(1 - 0,5)$ olur ve yeniden ıslatma süreyi uzatır. Bir kabusun korku enerjisi 0 veya altına indiğinde kabus dağılır ve ödülü oyuncuya eklenir. Şekil 4, (3) formülünün zırha göre net hasar üzerindeki etkisini göstermektedir.

Kod 2. Zırh formülü ve okcunun zırhlı cezası

```
1 // OyuncakSavunucu: PDF Bölüm 4.3
2 public static double zırhFormuluUygula(double hasar, int
   zırh) {
3     return hasar * (1 - (zırh / (zırh + 100.0)));
4 }
5 public final double netHasarHesapla(KabusYaratigi hedef) {
6     return zırhFormuluUygula(tabanHasarHesapla(hedef),
7                               hedef.getZırhKaplamasi());
8 }
9 // LastikSapan: taban hasarı ezer (polimorfizm)
10 @Override
11 protected double tabanHasarHesapla(KabusYaratigi hedef) {
12     return hedef.zırhlıMi() ? getVurusGucu() * 0.5 :
13         getVurusGucu();
14 }
```



Şekil 4. Zırh formülüne göre isabet başına net hasar. Zırh 100 olduğunda taban hasarın yarısı, zırh 50 olduğunda üçte ikisi hedefe ulaşır.



Şekil 5. Gece Yarısı Koridoru haritasının modeli: yol (ok yönünde) ve numaralı oyuncak yuvaları.

F. Harita Modeli

Harita, mantıksal olarak 880×560 piksellik bir düzlemde tanımlanmış ve ekran boyutuna göre oranı korunarak ölçeklenmektedir. Yol, dolaptan (başlangıç) yatağa (üs) uzanan 10 köşeli, 1982px uzunluğunda kesintisiz bir yılan biçimindedir (Şekil 5). On adet oyuncak yuvası, yolun kıvrımlarının iç ve dış kısımlarına, farklı uzunluktaki yol kesimlerini kapsayacak biçimde yerleştirilmiştir. Harita sınıfı oluşturulurken her yuvanın yola belirli bir mesafeden daha yakın olmadığı denetlenir; böylece savunucuların yolun üzerine kurulması engellenir.

G. Grafik Arayüz ile Simülasyon Motorunun Haberleşmesi

Arayüz ile motor arasındaki ilişki, motorun arayüzden habersiz kalacağı biçimde kurgulanmıştır. Haberleşme üç kanaldan yürür (Şekil 6):

1) *Komut kanalı (arayüz → motor):* Oyuncu bir savunucu düğmesine ve ardından bir yuvaya tıkladığında OyunEkrani, motorun savunucuInsaEt(tur, yuva) metodunu çağırır. Motor şeker, yuvanın doluluğunu ve oyunun bitip bitmediğini denetler, sonucu InsaSonucu enum'ı (BASARILI, YETERSİZ_SEKER, YUVA_DOLU, OYUN_BITTI) olarak döndürür. Arayüz bu sonuca göre alttaki ipucu satırında geri bildirim gösterir. Akını başlatma, duraklatma ve hızlandırma da aynı şekilde metot çağrılarıyla yapılır. Para yetmediğinde

düğmeler pasif hâle geldiği için geçersiz komut çoğunlukla arayüzde engellenir; yine de son denetim her zaman motordadır.

2) *Oyun döngüsü (zamanlayıcı ile çekme)*: OyunEkranı içinde 16 ms aralıklı bir `javax.swing.Timer` [8] çalışır (yaklaşık 60 kare/sn). Her tikte geçen gerçek süre `System.nanoTime()` ile ölçülür, takımlarda büyük sıçramaları önlemek için en fazla 0,05 sn ile sınırlanır ve `motor.guncelle(dt)` çağrılır (Kod 3). Hız $\times 2$ seçildiğinde aynı karede iki adım işletilir. Ardından üst bilgi çubuğu (uyku, şeker, akın) motordan okunarak yenilenir ve `HaritaPaneli.repaint()` çağrılır. Harita paneli çizim sırasında motordan kabus, savunucu, mermi ve efekt listelerini okur ve her nesnenin `ciz()` metodunu çağırır. Swing zamanlayıcısının olayları olay dağıtım iş parçacığında (EDT) çalıştığı için motor güncellemesi ile çizim aynı iş parçacığında sırayla yürür; ek bir kilit mekanizmasına gerek kalmaz [7].

Kod 3. Oyun döngüsünün bir karesi (OyunEkranı)

```
1 private void kare() { // javax.swing.Timer, 16 ms
2     long simdi = System.nanoTime();
3     double dt = Math.min(0.05, (simdi - sonZaman) / 1e9);
4     sonZaman = simdi;
5     for (int i = 0; i < hizCarpani; i++) motor.guncelle(dt);
6     paneliGuncelle(); // uyku, şeker, akın, düş
7     haritaPaneli.repaint(); // çiz() çağrıları
8 }
```

3) *Olay bildirimi (motor → arayüz)*: Motor, arayüzün hangi sınıf olduğunu bilmez; yalnızca `OyunOlayDinleyicisi` arayüzünü tanır (Gözlemci deseni [3]). Kayıt dosyasına yazılan her satır `gunlukSatiriEklendi()` ile, oyunun sonu ise `oyunBitti(kazanildi)` ile bildirilir. `OyunEkranı` bu arayüzü gerçekleştirerek satırları sağdaki canlı günlük paneline ekler; oyun bittiğinde haritada büyük bir KAZANDINIZ/KAYBETTİNİZ yazısı gösterir ve kısa bir süre sonra *Tekrar oyna* ve *Ana menü* seçeneklerini sunar.

H. Kayıt Sistemi

`SavunmaGunlugu` sınıfı, program başladığında çalışma dizininde `savunma_gunlugu.txt` dosyasını UTF-8 kodlamasıyla açar. Her satır `[SS:dd:ss.mmm T+saniye]` olay biçimindedir: ilk alan gerçek saat, ikinci alan simülasyon zamanıdır. Simülasyon zamanı duraklatmada durduğu ve hızlandırmada iki kat ilerlediği için olayların oyun içindeki sırası her koşulda doğru izlenebilir. Her satır yazıldıktan sonra dosya boşaltılır (*flush*); böylece program beklenmedik biçimde kapansa bile kayıtlar kaybolmaz. Dosya açılmazsa oyun durmaz, kayıtlar yalnızca ekrandaki panele yazılır ve kullanıcı uyarılır. Kaydedilen olaylar: simülasyonun başlaması, oyun başında oluşturulan iki akının tam içeriği (kimlik ve zırh değerleriyle), başarılı ve başarısız savunucu kurma girişimleri, akın başlangıcı, her kabusun dolaptan çıkışı, hedef değişimleri, her isabetin adım adım hasar hesabı, ıslatma, alan vuruşu, uçanı atlama, dağılma ve ödül, yatağa ulaşma ve uyku kaybı, akın sonu ve oyun sonucudur.

I. Doğrulama Yaklaşımı

Akın içeriği `AkinYoneticisi` kurucusunda oluşturulduktan hemen sonra `kurallariDogrula()` metodu ile

denetlenir: birinci akının tam olarak iki standart, bir zırhlı ve bir uçan kabustan oluştuğu, ikinci akının 5–10 kabus içerdiği, her akında her türden en az bir kabus bulunduğu ve tüm zırh değerlerinin [50, 100] aralığında olduğu doğrulanır; bir kural bozulursa program başlamadan hata verir. Motor, arayüzden bağımsız olduğu için sabit rastgele tohum (*seed*) ve sabit adım ($\Delta t = 1/60$ sn) ile çalıştırılan bir test sınıfı yazılmış; Bölüm IV'teki senaryolar bu sayede birebir tekrarlanabilir hâle getirilmiştir.

III. OYUN KONSEPTİ

A. Konseptin Gerekçesi

Seçilen tema, *gece yarısı kabuslarına karşı oyuncak savunmasıdır*: Bir çocuk yatağında uyurken dolabın kapısı aralanır ve kabuslar halının üzerinden yatağa doğru ilerler. Çocuğun oyuncakları onu korumak için nöbettedir. Bu tema üç nedenle seçilmiştir. Birincisi, kule savunmanın soyut kavramları (üs, can, para, kule, dalga) evrensel bir çocukluk deneyiminde doğal karşılıklar bulmaktadır: üs *yatak*, can *uykunun derinliği*, dalga ise *gece yarısı akınıdır*. İkincisi, tema yaygın ortaçağ ya da uzay temalarından uzak olduğu için özgünlük isterini güçlü biçimde karşılamaktadır. Üçüncüsü, oyuncak ve kabus dünyası her düşman ve kule türüne hem görsel hem de mantıksal olarak inandırıcı bir kimlik kazandırmaya elverişlidir.

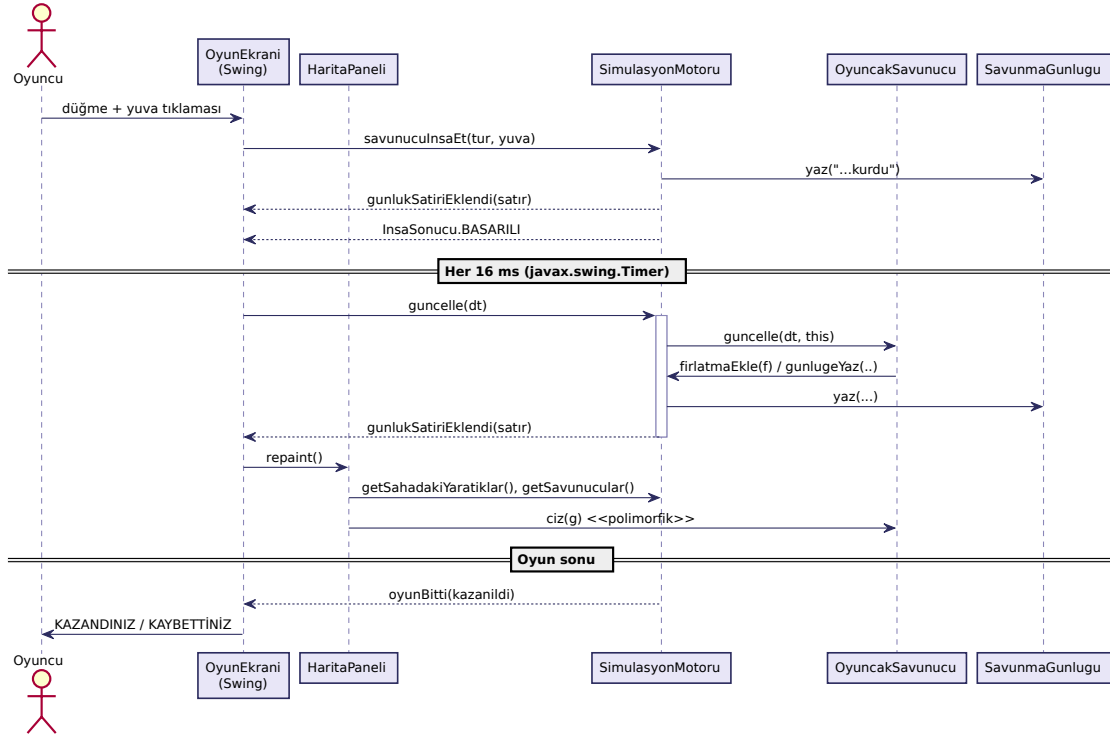
B. Konseptin Oyun Mekaniklerine Yansıması

Temanın yalnızca isimlerde kalmaması, her mekaniğin tema içinde bir anlam taşıması hedeflenmiştir:

- *Zırhlı düşman*: Teneke Robot, metal gövdesi nedeniyle hem yavaş hem de dayanıklıdır. Lastik Sapan'ın taşları teneke gövdeden sekerek daha az hasar verir; bu, okçunun zırhlıya %50 cezasının tema içindeki karşılığıdır.
- *Uçan düşman*: Gece Kelebeği odanın içinde uçarak ilerler. Bilye Mancınığı'nın bilyeleri yerde yuvarlandığı için kelebeğe ulaşamaz; topçunun uçanları hedefleyememesi böylece kendiliğinden anlamlıdır.
- *Alan hasarı*: Yere düşen bilyeler çevreye saçılır ve yakındaki yer kabuslarını da etkiler.
- *Yavaşlatma*: Buz kulesinin dondurma etkisi, Su Tabancası'nın *ıslatması* olarak yorumlanmıştır. Sırılsıklam olan kabus ağırlaşır yavaşlar ve ekranda mavi tonda, üzerinden su damlaları düşerek çizilir.
- *Ekonomi ve can*: Para birimi *şekerdir*; dağılan her kabus şeker bırakır. Yatağa ulaşan her kabus çocuğun uykusunu hafifletir; uyku tamamen kaçtığında çocuk uyanır ve oyun kaybedilir.

C. İsimlendirme Stratejisi

İsimlendirmede üç kural izlenmiştir: (1) Sınıf, alan ve metot adları temanın dünyasından seçilmiş Türkçe adlardır; (2) her ad, dokümandaki karşılığının işlevini açıkça hissettirmelidir (ör. `zırhKaplama`, `gorusMenzili`), böylece temaya rağmen kod okunabilir kalır; (3) Java adlandırma kuralları korunmuş, sınıflar `PascalCase`, alan ve metotlar `camelCase`, sabitler `BUYUK_HARF` ile yazılmıştır. Kodda Türkçeye özgü karakterler kullanılmamış (*GeceKelebegi*), arayüzde ve



Şekil 6. Arayüz ile simülasyon motoru arasındaki haberleşmenin sıra diyagramı: savunucu kurma komutu, her karede tekrarlanan güncelleme-çizim döngüsü ve oyun sonu bildirimi.

kayıt dosyasında ise doğru Türkçe yazım (*Gece Kelebeği*) kullanılmıştır. Tablo III başlıca eşleşmeleri vermektedir.

Kayıt dosyasında da aynı dil kullanılmıştır. Örneğin dokümandaki “*Dalga 1 Başladı*” olayı “*Gece Yarısı Akını 1 Başladı! Dolap kapısı aralandı.*”, düşmanın ölmesi “*dağıldı (yenildi)*”, üsse ulaşması ise “*yatağa ulaştı! Uyku Derinliği (Can): 90 (-5)*” biçiminde kaydedilmektedir.

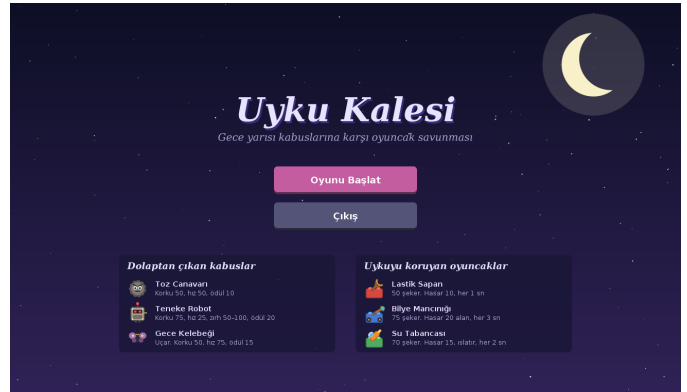
D. Görsel Tasarım Dili

Arayüzün tamamı gece temasına uygun koyu lacivert ve mor tonlarla tasarlanmış, tüm varlıklar harici görsel dosyası kullanılmadan Java2D ile vektörel olarak çizilmiştir (Şekil 7, Şekil 8). Harita; ahşap parke zemin, pencereden süzülen ay ışığı, dolaptan yatağa uzanan ve yönü oklarla belirtilen bir halı yol ile uyuyan çocuğun bulunduğu yataktan oluşur. Kabuslar türlerine göre ayırt edilebilir silüetlere (toz yumağı, teneke robot, kelebek) ve renklere sahiptir. Savunucular oyuncak bloklarının üzerinde durur ve hedefe doğru döner. Renkler işlevsel olarak da kullanılmıştır: şeker sarı, uyku mor, ıslaklık mavi, uygun yuva yeşil, uygun olmayan yuva kırmızı, pasif düğme gridir.

IV. SIMÜLASYON

A. Akın Yapılandırması

Dokümana uygun olarak tüm kabus nesneleri oyun başında, *AkinYoneticisi* kurucusunda oluşturulur ve dolapta bekletilir (Tablo IV). Birinci akın tam olarak iki standart, bir zırhlı ve bir uçan kabustan oluşur. İkinci akın ise on kabusluk, zırhlı



Şekil 7. Ana menü: *Oyunu Başlat* ve *Çıkış* düğmeleri ile kabus ve savunucu türlerinin özet bilgileri.

ağırlıklı bir akındır. İki akının toplam uyku hasarı $25 + 75 = 100$ olarak seçilmiştir; dolayısıyla hiç savunucu kurmayan bir oyuncu oyunu tam olarak kaybeder, bu da kaybetme koşulunun sunumda kolayca gösterilebilmesini sağlar.

B. Senaryolar ve Sonuçlar

Motorun doğruluğunu ve isterlerin karşılandığını göstermek için aynı rastgele tohumla (dolayısıyla aynı zırh değerleriyle) üç senaryo çalıştırılmıştır:

- *S1 – Savunmasız:* Hiç savunucu kurulmaz.
- *S2 – Dengeli savunma:* Başlangıçta 1 numaralı yuvaya Lastik Sapan, 4 numaralı yuvaya Su Tabancası, 6 numaralı

Tablo III
DOKÜMAN KAVRAMLARI İLE TEMADAKİ KARŞILIKLARI

Doküman	Tema	Kod	Doküman	Tema	Kod
Düşman (soyut)	Kabus yaratığı	KabusYaratigi	Hız / Zırh	Adım hızı / Zırh kaplaması	temelAdimHizi, zirhKaplamasi
Standart düşman	Toz Canavari	TozCanavari	Kule hasarı / menzil	Vuruş gücü / Görüş menzili	vurusGucu, gorusMenzili
Zırhlı düşman	Teneke Robot	TenekeRobot	Oyuncu canı	Uyku derinliği	uykuDerinligi
Uçan düşman	Gece Kelebeği	GeceKelebegi	Para	Şeker	seker, sekerMaliyeti
Kule (soyut)	Oyuncak savunucu	OyuncakSavunucu	Dalga	Gece Yarısı Akını	AkinYoneticisi
Okçu Kulesi	Lastik Sapan	LastikSapan	İnşa alanı	Oyuncak yuvası	OyuncakYuvasi
Topçu Kulesi	Bilye Mancınığı	BilyeMancinigi	Yavaşlatma	İslatma	islat(), islakMi()
Buz Kulesi	Su Tabancası	SuTabancasi	Ölmek / Üsse ulaşmak	Dağılmak / Yatağa ulaşmak	DAGILDI, YATAGA_ULASTI
Düşman canı	Korku enerjisi	korkuEnerjisi	Kayıt dosyası	Savunma günlüğü	SavunmaGunlugu



Şekil 8. İkinci akın sırasında oyun ekranı: üstte uyku, şeker ve akın bilgisi; ortada canlı harita; sağda savunma günlüğü; altta maliyetleriyle savunucu düğmeleri (şeker yetersiz olduğu için pasif).

Tablo IV
GECE YARISI AKINLARININ İÇERİĞİ

Akın	Toz C.	Teneke R.	Gece K.	Toplam	Uyku hasarı
1	2	1	1	4	25
2	3	5	2	10	75

Kimlikler: Akın 1 için ID101–ID104, Akın 2 için ID201–ID210.

yuvaya Bilye Mancınığı kurulur; akın arasında kazanılan şekerle 8 numaralı yuvaya bir Lastik Sapan daha eklenir. Ayrıca dolu yuvaya ve yetersiz şekerle kurma girişimleri bilerek yapılmıştır.

- S3 – Yalnızca topçu: 1 numaralı yuvaya tek bir Bilye Mancınığı kurulur.

Sonuçlar Tablo V’da, uyku derinliğinin zaman içindeki değişimi Şekil 10’da verilmiştir. Savunmasız senaryoda on dört kabusun tamamı yatağa ulaşmış, uyku tam olarak sıfıra inmiş ve oyun 188,5. saniyede KAYBETTİNİZ ile bitmiştir. Dengeli savunmada hiçbir kabus yatağa ulaşamamış, oyun 146,2. saniyede KAZANDINIZ ile sonuçlanmıştır. Yalnızca topçu kulan senaryoda Bilye Mancınığı üç Gece Kelebeğini de hedefleyemeyip atlamış, bu durum kayda geçmiş ve yalnızca üç kabus dağıtılabilmıştır; buna karşın uyku sıfıra inmediği için oyun kazanılmış, ancak uykunun %85’i kaybedilmiştir. Bu sonuç, uçan düşmana karşı topçunun etkisizliğini ve kule çeşitliliğinin önemini açıkça göstermektedir.

Tablo VI, (3) formülünün Teneke Robot üzerindeki etkisini sayısal olarak göstermektedir. Lastik Sapan zırhlı cezası ve



(a) Savunucu yerleştirme: uygun yuvalar yeşil, menzil önzilemesi ve bilgi kutusu.



(b) Su Tabancası ile ıslatılıp yavaşlatılan Toz Canavarı (ID209) mavi çizilir.



(c) KAZANDINIZ: tüm akınlar püskürtüldü.

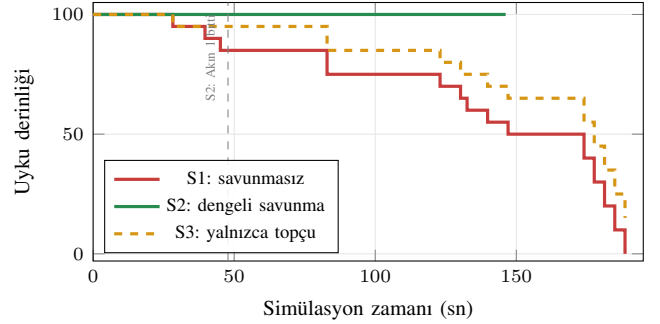


(d) KAYBETTİNİZ: uyku derinliği sıfıra indi.

Tablo V
SENARYO SONUÇLARI (SABİT TOHUM, $\Delta t = 1/60$ SN)

	S1	S2	S3
Kurulan savunucu	0	4	1
Başarısız kurma girişi	0	4	0
İsabet (tek hedef) sayısı	0	103	0
Alan vuruşu (merkez/yakın)	0/0	11/4	22/3
Islatma sayısı	0	21	0
Atlanan uçan kabus	—	0	3
Dağılan kabus	0	14	3
Yatağa ulaşan kabus	14	0	11
Kalan uyku derinliği	0	100	15
Oyun sonu şekeri	200	170	155
Bitiş zamanı (sn)	188,5	146,2	188,5
Sonuç	KAYBETTİNİZ	KAZANDINIZ	KAZANDINIZ

zırh formülü birlikte uygulandığında zırhlı hedefe isabet başına yalnızca 2,5–3,3 hasar verebilmektedir; bu nedenle zırhlı kabuslara karşı Bilye Mancınığı ve Su Tabancası çok daha verimlidir. Bu bulgu, kayıt dosyasındaki “Taban 10 → Zırhlı cezası %50 → 5; Zırh formülü (Zırh 76) ile Net Hasar 2.84” gibi satırlarla birebir doğrulanmıştır.



Şekil 10. Üç senaryoda uyku derinliğinin (oyuncu canı) simülasyon zamanına göre değişimi.

C. Kayıt Dosyası Çıktıları

Kod 4, S2 senaryosunda üretilen savunma_gunlugu.txt dosyasından seçilmiş satırları göstermektedir. Satırlarda oyun başında oluşturulan akınlar, başarılı ve başarısız kurma girişimleri, akının başlaması, kabusun dolaptan çıkışı, hedef seçimi, zırhlı hedefe adım adım hasar hesabı, ıslatma, alan

Kod 4. S2 senarvosunda üretilen savunma gunluugu.txt dosvasından secilmis satırlar

```
[12:50:29.007 | T+000.00s] Simülasyon Başladı. Senaryo: 'Gece Yarısı Koridoru'. Başlangıç Uyku Derinliği (Can): 100, Şeker
↳ (Para): 200.
[12:50:29.105 | T+000.00s] Dolapta hazırlanan Gece Yarısı Akını 1: TozCanavari x2, GeceKelebegi x1, TenekeRobot x1; Toplam:
↳ 4 → TozCanavari-ID101, GeceKelebegi-ID102, TenekeRobot-ID103 (Zırh 76), TozCanavari-ID104.
[12:50:29.218 | T+000.00s] Kullanıcı, (170, 175) konumuna (Oyuncak Yuvası #1) 'LastikSapan-ID001' kurdu. Maliyet: 50.
↳ Kalan Şeker: 150.
[12:50:29.243 | T+000.00s] Kullanıcı, (330, 375) konumundaki Oyuncak Yuvası #4 için Su Tabancası istedi; yuva dolu,
↳ kurulmadı.
[12:50:29.244 | T+000.00s] Kullanıcı, (510, 300) konumuna (Oyuncak Yuvası #6) 'BilyeMancinigi-ID003' kurdu. Maliyet: 75.
↳ Kalan Şeker: 5.
[12:50:29.266 | T+000.00s] Kullanıcı, (170, 355) konumuna Bilye Mancınığı istedi; şeker yetersiz (5/75), kurulmadı.
[12:50:29.270 | T+000.00s] Gece Yarısı Akını 1 Başladı! Dolap kapısı aralandı. (Dağılım: TozCanavari x2, GeceKelebegi x1,
↳ TenekeRobot x1; Toplam: 4)
[12:50:29.276 | T+000.02s] Kabus 'TozCanavari-ID101' dolaptan çıktı (Korku Enerjisi: 50/50, Zırh: 0, Hız: 50 px/sn, yerde).
↳
[12:50:29.712 | T+000.45s] Savunucu 'LastikSapan-ID001' hedef aldı: 'TozCanavari-ID101' (öncelik: yatağa en yakın, kalan
↳ yol 1960 px).
[12:50:44.064 | T+014.65s] Sapan taşı isabet etti: 'TenekeRobot-ID103' ← LastikSapan-ID001: Taban 10 → Zırhlı cezası %50
↳ → 5; Zırh formülü (Zırh 76) ile Net Hasar 2.84. Kalan Korku Enerjisi: 72.16/75.
[12:51:09.542 | T+039.80s] Su jeti isabet etti: 'TenekeRobot-ID103' ← SuTabancası-ID002: Taban 15; Zırh formülü (Zırh 76)
↳ ile Net Hasar 8.52. Kalan Korku Enerjisi: 32.39/75.
[12:51:09.543 | T+039.80s] 'TenekeRobot-ID103' sırlıslıkam oldu: Yavaşlatma %50 (3 sn), hız 25 → 12.5 px/sn.
[12:52:21.694 | T+111.07s] Savunucu 'BilyeMancinigi-ID003' bilyesi yere çarptı (uçanları hedefleyemez). Alan vuruşu:
↳ merkez 'TenekeRobot-ID202', yarıçap 50 px.
[12:52:21.695 | T+111.07s] -- Merkez: 'TenekeRobot-ID202' ← BilyeMancinigi-ID003: Taban 20; Zırh formülü (Zırh 95) ile
↳ Net Hasar 10.26. Kalan Korku Enerjisi: 0/75.
[12:52:21.696 | T+111.07s] -- Yakındaki: 'TenekeRobot-ID204' ← BilyeMancinigi-ID003: Taban 20; Zırh formülü (Zırh 50) ile
↳ Net Hasar 13.33. Kalan Korku Enerjisi: 35/75.
[12:50:34.009 | T+004.70s] 'TozCanavari-ID101' dağıldı (yenildi). Ödül +10 şeker. Toplam Şeker: 15.
[12:51:17.726 | T+047.80s] Gece Yarısı Akını 1 püskürtüldü. Uyku: 100, Şeker: 60.
[12:51:17.726 | T+047.80s] Sonraki akına 10 sn var; savunmayı güçlendirme zamanı.
...
[12:52:57.255 | T+146.22s] Gece Yarısı Akını 2 püskürtüldü. Uyku: 100, Şeker: 170.
[12:52:57.257 | T+146.22s] SON: Tüm akınlar püskürtüldü, haritada kabus kalmadı. OYUN KAZANILDI -- KAZANDINIZ! (Kalan Uyku:
↳ 100, Toplam Şeker: 170)
```

Kod 5. S3 ve S1 senarvolarından secilmis kayıt satırları: topcunun ucanı atlaması. yatağa ulasma ve kaybetme

```
# S3 (yalnızca topçu)
[12:50:32.976 | T+003.70s] Savunucu 'BilyeMancinigi-ID001' uçan 'GeceKelebegi-ID102' yarattığını hedefleyemez; atladı (
↳ bilyeler yerde yuvarlanır).
[12:50:30.383 | T+001.13s] -- Merkez: 'TozCanavari-ID101' ← BilyeMancinigi-ID001: Taban 20; zırh yok, Net Hasar 20. Kalan
↳ Korku Enerjisi: 30/50.
[12:50:57.800 | T+028.25s] 'GeceKelebegi-ID102' yatağa ulaştı! Uyku Derinliği (Can): 95 (-5).
# S1 (savunmasız)
[12:50:57.746 | T+028.25s] 'GeceKelebegi-ID102' yatağa ulaştı! Uyku Derinliği (Can): 95 (-5).
[12:51:53.173 | T+082.92s] 'TenekeRobot-ID103' yatağa ulaştı! Uyku Derinliği (Can): 75 (-10).
...
[12:53:40.011 | T+188.53s] 'TenekeRobot-ID210' yatağa ulaştı! Uyku Derinliği (Can): 0 (-10).
[12:53:40.012 | T+188.53s] SON: Çocuk kabuslarla uyandı! OYUN KAYBEDİLDİ -- KAYBETTİNİZ. (Kalan Uyku: 0, Toplam Şeker: 200)
↳
```

Tablo VI

TENEKE ROBOT (75 KORKU ENERJİSİ) İÇİN NET HASAR VE GEREKEN
ISABET SAYISI

Savunucu	Z = 50		Z = 75		Z = 100	
	H_{net}	isabet	H_{net}	isabet	H_{net}	isabet
Lastik Sapan	3,33	23	2,86	27	2,50	30
Bilye Mancınığı	13,33	6	11,43	7	10,00	8
Su Tabancası	10,00	8	8,57	9	7,50	10

vuruşu, dağılma ve ödül ile oyunun sonucu görülmektedir. Kod 5 ise S3 ve S1 senaryolarından topcunun ucanı atlaması, yatağa ulaşma ve kaybetme kayıtlarını içermektedir.

D. Canlı Görselleştirme

Şekil 8, ikinci akın sırasında alınmış tam oyun ekranını göstermektedir. Üst bilgi çubuğunda uyku derinliği (sayı ve çubuk), şeker ve akın bilgisi (aktif akın, sahadaki ve dolapta

bekleyen kabus sayısı) her an görünür. Akın sürerken akın başlatma düğmesi pasiftir. Sağdaki panel, kayıt dosyasına yazılan satırları canlı olarak gösterir. Alttaki savunucu düğmeleri dokümandaki biçimde, maliyetiyle birlikte (*Lastik Sapan [50 şeker]*) gösterilir ve şeker yetmediğinde gri ve tıklanamaz hâle gelir; şekilde 45 şeker olduğu için üç düğme de pasiftir. Her kabusun üzerinde kimliği, zırhlıysa zırh değeri (ör. Z62) ve sağlık barı; her savunucunun altında ise kimliği ve yeniden doldurma çubuğu bulunur. İsabet anında kabus beyaz renkte parlar ve üzerinde kırmızı hasar sayısı belirir; dağılan kabusun yerinde kazanılan şeker miktarı yükselir.

Şekil 9(a)'da oyuncu Bilye Mancınığı'nı seçip boş bir yuvanın üzerine geldiğinde görülen arayüz yer almaktadır: boş yuvalar yeşil çerçeveye vurgulanır, imlecin altındaki yuva için savunucunun menzili yarı saydam bir daire olarak önizlenir ve savunucunun özellikleri bir bilgi kutusunda gösterilir. Şekil 9(b)'de ise Su Tabancası tarafından ısıtılan bir Toz Canavari mavi tonda görülmektedir.

Şekil 9(c) ve (d), oyunun iki olası sonucunu göstermektedir. Kazanma durumunda tüm akınlar püskürtülmüş ve haritada kabus kalmamıştır; kaybetme durumunda ise uyku derinliği sıfıra inmiştir. Her iki durumda da harita karartılır, sonuç büyük yazıyla gösterilir ve kayıt dosyasına son satır olarak yazılır.

V. SONUÇ

Bu çalışmada proje dokümanındaki tüm isterleri karşılayan, temalı ve gerçek zamanlı bir kule savunma oyunu geliştirilmiştir. Üç düşman ve üç kule türü, soyut sınıflar üzerine kurulu iki hiyerarşi ile modellenmiş; ortak davranışlar kalıtımla bir kez yazılmış, türlere özgü davranışlar ise metot ezme ile çok biçimli olarak çözülmüştür. Şablon metot, fabrika ve gözlemci desenleri kodun okunabilirliğini ve genişletilebilirliğini artırmıştır: yeni bir kabus ya da savunucu türü eklemek için yalnızca yeni bir alt sınıf yazmak ve gerekirse *SavunucuTuru* enum'ına bir sabit eklemek yeterlidir.

Simülasyon motorunun grafik arayüzden tamamen ayrılması, projenin en önemli tasarım kararlarından biri olmuştur. Bu ayırım sayesinde motor arayüz olmadan, sabit tohum ve sabit zaman adımıyla test edilebilmiş; kazanma ve kaybetme koşulları, zırh formülü, hedef önceliği, alan hasarı ve yavaşlatma etkisi tekrarlanabilir senaryolarla doğrulanmıştır. Ayrıntılı kayıt dosyası, her hasar değerinin nasıl hesaplandığını adım adım gösterdiği için hem hata ayıklamada hem de isterlerin denetlenmesinde etkili bir araç olmuştur.

Senaryo sonuçları, kule çeşitliliğinin önemini sayısal olarak ortaya koymuştur: yalnızca topçu kullanıldığında uçan düşmanlar ve dayanıklı zırhlılar savunmayı aşarken, farklı kule türlerinin birlikte kullanıldığı savunma hiçbir kabusu yatağa ulaştırmamıştır. Gelecek çalışmalarda savunucuların yükseltilmesi ve satılması, birden fazla harita ve zorluk seviyesi, ses efektleri ile en yüksek skor tablosu eklenebilir; ayrıca motorun ayrık yapısı, dalga yapılandırmasının dış bir dosyadan okunmasına da elverişlidir.

YAZAR KATKILARI

Ad SOYAD (1. yazar): Oyun konseptinin ve isimlendirme stratejisinin belirlenmesi, *varlık* paketindeki sınıf hiyerarşisinin (kabuslar, savunucular, hasar ve etki mekanikleri) tasarımı ve gerçekleştirilmesi, UML diyagramlarının hazırlanması, raporun Yöntem ve Oyun Konsepti bölümlerinin yazımı.

Ad SOYAD (2. yazar): Simülasyon motoru, akın yöneticisi ve kayıt sisteminin gerçekleştirilmesi, Swing arayüzünün (ana menü, oyun ekranı, harita çizimi) tasarımı, senaryo testlerinin yürütülmesi, ekran görüntülerinin ve kayıt çıktılarının hazırlanması, raporun Simülasyon ve Sonuç bölümlerinin yazımı.

KAYNAKÇA

- [1] Kocaeli Üniversitesi Bilgisayar Mühendisliği Bölümü, "Programlama Laboratuvarı-I, Proje II: Kule Savunma Oyunu," proje dokümanı, Kocaeli, Türkiye, 2025.
- [2] P. Avery, J. Togelius, E. Alistar, and R. P. van Leeuwen, "Computational intelligence and tower defence games," in *Proc. IEEE Congr. Evol. Comput. (CEC)*, New Orleans, LA, USA, 2011, pp. 1084–1091.
- [3] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.

- [4] R. Nystrom, *Game Programming Patterns*. Genever Benning, 2014. [Çevrimiçi]. Erişim: <https://gameprogrammingpatterns.com>
- [5] J. Bloch, *Effective Java*, 3rd ed. Boston, MA, USA: Addison-Wesley, 2018.
- [6] Oracle, "Creating a GUI With Swing," *The Java Tutorials*. [Çevrimiçi]. Erişim: <https://docs.oracle.com/javase/tutorial/uiswing/>
- [7] Oracle, "Concurrency in Swing," *The Java Tutorials*. [Çevrimiçi]. Erişim: <https://docs.oracle.com/javase/tutorial/uiswing/concurrency/>
- [8] Oracle, "Class `javax.swing.Timer`," *Java SE 11 API Specification*. [Çevrimiçi]. Erişim: <https://docs.oracle.com/en/java/javase/11/docs/api/java.desktop/javax/swing/Timer.html>
- [9] Object Management Group, *OMG Unified Modeling Language (OMG UML), Version 2.5.1*, OMG Standard formal/17-12-05, Dec. 2017.
- [10] PlantUML. [Çevrimiçi]. Erişim: <https://plantuml.com>